

# ❑ 50 PRINCIPAIS COMANDOS DO SELENIUM PARA RPA

## ❑ CATEGORIAS DOS COMANDOS:

1. Configuração e Inicialização
2. Navegação e URLs
3. Localização de Elementos (Find Elements)
4. Interação com Elementos
5. Esperas (Waits)
6. Janelas e Abas
7. Alertas e Pop-ups
8. Frames e Iframes
9. Cookies e Storage
10. JavaScript Execução
11. Capturas e Screenshots
12. Ações Avançadas
13. Informações e Estado

## ❑ 50 COMANDOS ESSENCIAIS

### 1. CONFIGURAÇÃO E INICIALIZAÇÃO

```
python
```

```
# 1. Driver Chrome com webdriver-manager (RECOMENDADO)
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
```

```
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)
```

```
# 2. Opções do Chrome
```

```
from selenium.webdriver.chrome.options import Options
options = Options()
options.add_argument('--headless') # Execução em background
options.add_argument('--start-maximized')
driver = webdriver.Chrome(options=options)

# 3. Driver Firefox
from selenium.webdriver import Firefox
from webdriver_manager.firefox import GeckoDriverManager
driver = Firefox(executable_path=GeckoDriverManager().install())

# 4. Fechar driver
driver.quit() # Fecha tudo
driver.close() # Fecha aba atual
```

## 2. NAVEGAÇÃO E URLS

python

```
# 5. Acessar URL
driver.get("https://www.exemplo.com")

# 6. Navegar para frente/trás
driver.forward()
driver.back()

# 7. Atualizar página
driver.refresh()

# 8. Obter URL atual
current_url = driver.current_url

# 9. Obter título da página
page_title = driver.title

# 10. Obter código fonte
page_source = driver.page_source
```

## 3. LOCALIZAÇÃO DE ELEMENTOS (BY)

python

```
from selenium.webdriver.common.by import By

# 11. Por ID
element = driver.find_element(By.ID, "username")

# 12. Por NAME
element = driver.find_element(By.NAME, "email")

# 13. Por CLASS_NAME
element = driver.find_element(By.CLASS_NAME, "form-control")

# 14. Por TAG_NAME
element = driver.find_element(By.TAG_NAME, "input")

# 15. Por LINK_TEXT (texto exato do link)
element = driver.find_element(By.LINK_TEXT, "Clique Aqui")

# 16. Por PARTIAL_LINK_TEXT (texto parcial)
element = driver.find_element(By.PARTIAL_LINK_TEXT, "Clique")

# 17. Por XPATH (MAIS PODEROSO)
element = driver.find_element(By.XPATH, "//input[@id='username']")

# 18. Por CSS_SELECTOR (mais rápido)
element = driver.find_element(By.CSS_SELECTOR, "#username")

# 19. Múltiplos elementos
elements = driver.find_elements(By.CLASS_NAME, "produto") # Note: find_elements

# 20. Encontrar dentro de elemento
container = driver.find_element(By.ID, "formulario")
input_inside = container.find_element(By.NAME, "campo")
```

## 4. INTERAÇÃO COM ELEMENTOS

```
python
```

```
from selenium.webdriver.common.keys import Keys

# 21. Clicar em elemento
element.click()
```

```
# 22. Enviar texto
element.send_keys("texto de exemplo")

# 23. Limpar campo
element.clear()

# 24. Teclas especiais
element.send_keys(Keys.ENTER)
element.send_keys(Keys.TAB)
element.send_keys(Keys.BACKSPACE)
element.send_keys(Keys.CONTROL + 'a') # Selecionar tudo

# 25. Obter texto do elemento
text = element.text

# 26. Obter atributo
value = element.get_attribute("value")
href = element.get_attribute("href")
class_list = element.get_attribute("class")

# 27. Verificar se está habilitado
is_enabled = element.is_enabled()

# 28. Verificar se está selecionado (checkboxes/radio)
is_selected = element.is_selected()

# 29. Verificar se está visível
is_displayed = element.is_displayed()

# 30. Submeter formulário
element.submit()
```

## 5. ESPERAS (WAITS) - CRÍTICO PARA RPA

```
python
```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 31. Espera explícita - elemento clicável
wait = WebDriverWait(driver, 10)
element = wait.until(EC.element_to_be_clickable((By.ID, "botao")))
```

```
# 32. Espera - elemento visível
element = wait.until(EC.visibility_of_element_located((By.ID, "elemento")))

# 33. Espera - elemento presente (DOM)
element = wait.until(EC.presence_of_element_located((By.ID, "elemento")))

# 34. Espera - texto presente no elemento
wait.until(EC.text_to_be_present_in_element((By.ID, "status"), "Concluído"))

# 35. Espera - título da página
wait.until(EC.title_contains("Dashboard"))

# 36. Espera - alerta presente
alert = wait.until(EC.alert_is_present())

# 37. Espera implícita (global)
driver.implicitly_wait(10) # Segundos

# 38. Sleep (EVITAR quando possível)
import time
time.sleep(5) # Último recurso
```

## 6. JANELAS E ABAS

python

```
# 39. Obter janela atual
current_window = driver.current_window_handle

# 40. Obter todas as janelas
all_windows = driver.window_handles

# 41. Mudar para nova janela
driver.switch_to.window(all_windows[1])

# 42. Fechar janela atual e voltar
driver.close()
driver.switch_to.window(all_windows[0])

# 43. Executar em nova aba
```

```
driver.execute_script("window.open('');")
driver.switch_to.window(driver.window_handles[1])
```

## 7. ALERTAS E POP-UPS

python

```
# 44. Aceitar alerta
alert = driver.switch_to.alert
alert.accept()
```

```
# 45. Recusar/cancelar alerta
alert.dismiss()
```

```
# 46. Obter texto do alerta
alert_text = alert.text
```

```
# 47. Enviar texto para prompt
alert.send_keys("Resposta")
```

## 8. FRAMES E IFRAMES

python

```
# 48. Mudar para iframe por índice
driver.switch_to.frame(0)
```

```
# 49. Mudar para iframe por nome/ID
driver.switch_to.frame("frame_nome")
```

```
# 50. Voltar para conteúdo principal
driver.switch_to.default_content()
```

## ❑ COMANDOS AVANÇADOS PARA RPA

### AÇÕES COMPLEXAS (ActionChains)

python

```
from selenium.webdriver.common.action_chains import ActionChains

# 51. Clicar e segurar
actions = ActionChains(driver)
actions.click_and_hold(element).perform()

# 52. Duplo clique
actions.double_click(element).perform()

# 53. Clique direito (context menu)
actions.context_click(element).perform()

# 54. Arrastar e soltar
actions.drag_and_drop(source_element, target_element).perform()

# 55. Mover para elemento
actions.move_to_element(element).perform()

# 56. Teclas modificadoras
actions.key_down(Keys.CONTROL).send_keys('c').key_up(Keys.CONTROL).perform()
```

### EXECUTAR JAVASCRIPT

python

```
# 57. Executar JS genérico
driver.execute_script("alert('Hello Selenium!');")

# 58. Rolar página
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);") # Para baixo
driver.execute_script("window.scrollTo(0, 0);") # Para cima

# 59. Rolar até elemento
driver.execute_script("arguments[0].scrollIntoView();", element)
```

```
# 60. Clicar via JS (quando click() não funciona)
driver.execute_script("arguments[0].click();", element)

# 61. Alterar atributo via JS
driver.execute_script("arguments[0].setAttribute('value', 'novo valor')", element)
```

## GERENCIAMENTO DE COOKIES

python

```
# 62. Obter todos os cookies
cookies = driver.get_cookies()

# 63. Adicionar cookie
driver.add_cookie({'name': 'test', 'value': '123'})

# 64. Obter cookie específico
cookie = driver.get_cookie('session_id')

# 65. Deletar cookie
driver.delete_cookie('cookie_name')

# 66. Deletar todos cookies
driver.delete_all_cookies()
```

## INFORMAÇÕES DO NAVEGADOR

python

```
# 67. Obter tamanho da janela
size = driver.get_window_size()
width = size['width']
height = size['height']

# 68. Definir tamanho da janela
driver.set_window_size(1024, 768)

# 69. Maximizar janela
driver.maximize_window()
```



```
# 70. Minimizar janela
driver.minimize_window()

# 71. Obter posição da janela
position = driver.get_window_position()
x = position['x']
y = position['y']

# 72. Definir posição
driver.set_window_position(0, 0)
```

## CAPTURAS E SCREENSHOTS

python

```
# 73. Capturar tela inteira
driver.save_screenshot('screenshot.png')

# 74. Capturar elemento específico
element.screenshot('element_screenshot.png')

# 75. Capturar como base64 (para relatórios)
screenshot_base64 = driver.get_screenshot_as_base64()

# 76. Capturar como binário
screenshot_binary = driver.get_screenshot_as_png()
```

## SELECT (DROPDOWN)

python

```
from selenium.webdriver.support.ui import Select

# 77. Selecionar por valor
select = Select(driver.find_element(By.ID, "estado"))
select.select_by_value("SP")

# 78. Selecionar por texto visível
select.select_by_visible_text("São Paulo")

# 79. Selecionar por índice
```

```
select.select_by_index(1)

# 80. Deselecionar tudo (múltipla escolha)
select.deselect_all()

# 81. Obter opções
options = select.options
for option in options:
    print(option.text)

# 82. Obter opção selecionada
selected_option = select.first_selected_option
```

## GERENCIAMENTO DE TEMPO

python

```
# 83. Timeout de página
driver.set_page_load_timeout(30) # 30 segundos

# 84. Timeout de script
driver.set_script_timeout(20)

# 85. Obter logs do navegador
logs = driver.get_log('browser')
for log in logs:
    print(log['message'])
```

## LOCATORS AVANÇADOS (XPATH/CSS)

python

```
# 86. XPath - contains
//button[contains(text(), 'Salvar')]
//div[contains(@class, 'alert')]

# 87. XPath - starts-with
//input[starts-with(@id, 'user')]

# 88. XPath - combinando condições
//input[@type='text' and @name='email']
```

```
# 89. XPath - eixos
//div[@id='container']/input # Descendentes
//div[@id='container']/input # Filhos diretos
//label[text()='Nome:']/following-sibling::input
```

```
# 90. CSS Selector - avançado
input[type='submit']
#content > .form-group input
div.alert.alert-success
```

## TRATAMENTO DE ERROS

```
python
```

```
from selenium.common.exceptions import *
```

```
# 91. Try/except comum
try:
    element = driver.find_element(By.ID, "inexistente")
except NoSuchElementException:
    print("Elemento não encontrado")
```

```
# 92. Verificar se elemento existe
def element_exists(locator):
    try:
        driver.find_element(*locator)
        return True
    except NoSuchElementException:
        return False
```

## OTIMIZAÇÕES PARA RPA

```
python
```

```
# 93. Desabilitar imagens para performance
chrome_options.add_argument('--blink-settings=imagesEnabled=false')
```

```
# 94. Desabilitar JavaScript (cuidado!)
chrome_options.add_argument('--disable-javascript')
```

```
# 95. User-Agent personalizado
```

```

chrome_options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36')

# 96. Modo headless com resolução
chrome_options.add_argument('--headless')
chrome_options.add_argument('--window-size=1920,1080')

# 97. Ignorar certificados SSL
chrome_options.add_argument('--ignore-certificate-errors')

# 98. Desabilitar notificações
chrome_options.add_argument('--disable-notifications')

# 99. Linguagem do navegador
chrome_options.add_argument('--lang=pt-BR')

# 100. Extensão do Chrome
chrome_options.add_extension('extension.crx')

```

## ❑ EXEMPLO PRÁTICO COMPLETO PARA RPA

python

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.action_chains import ActionChains
from webdriver_manager.chrome import ChromeDriverManager
import time

class RPAAutomation:
    def __init__(self, headless=False):
        # Configurar driver
        service = Service(ChromeDriverManager().install())
        options = webdriver.ChromeOptions()

        if headless:
            options.add_argument('--headless')

```

```

options.add_argument('--start-maximized')
options.add_argument('--disable-notifications')

self.driver = webdriver.Chrome(service=service, options=options)
self.wait = WebDriverWait(self.driver, 15)
self.actions = ActionChains(self.driver)

def navigate_to(self, url):
    """Navegar para URL"""
    self.driver.get(url)

def fill_form(self, data):
    """Preencher formulário genérico"""
    # Texto
    self.wait_and_send_keys(By.ID, "nome", data['nome'])

    # Email
    self.wait_and_send_keys(By.NAME, "email", data['email'])

    # Dropdown
    select = Select(self.wait_for_element(By.ID, "cargo"))
    select.select_by_visible_text(data['cargo'])

    # Checkbox
    if data['aceito']:
        checkbox = self.wait_for_element(By.CSS_SELECTOR,
☐)
        if not checkbox.is_selected():
            checkbox.click()

    # Enviar
    submit = self.wait_for_element(By.XPATH, "//button[text()='Enviar']")
    submit.click()

def wait_for_element(self, by, value, timeout=15):
    """Esperar elemento ficar visível"""
    return self.wait.until(
        EC.visibility_of_element_located((by, value))
    )

def wait_and_click(self, by, value):
    """Esperar e clicar"""

```

```

        element = self.wait.until(
            EC.element_to_be_clickable((by, value))
        )
        element.click()

def wait_and_send_keys(self, by, value, text):
    """Esperar e enviar texto"""
    element = self.wait_for_element(by, value)
    element.clear()
    element.send_keys(text)

def upload_file(self, file_path):
    """Upload de arquivo"""
    file_input = self.driver.find_element(By.XPATH, "//input[@type='file']")
    file_input.send_keys(file_path)

def download_file(self, element):
    """Configurar download"""
    # Configurar diretório de download
    options = self.driver.capabilities['chrome']['args'] = {
        'download.default_directory': '/caminho/downloads'
    }
    element.click()

def take_screenshot(self, name="screenshot"):
    """Capturar screenshot"""
    timestamp = time.strftime("%Y%m%d_%H%M%S")
    filename = f"{name}_{timestamp}.png"
    self.driver.save_screenshot(filename)
    return filename

def switch_to_iframe(self, identifier):
    """Mudar para iframe"""
    self.driver.switch_to.frame(identifier)

def switch_to_default(self):
    """Voltar para conteúdo principal"""
    self.driver.switch_to.default_content()

def handle_alert(self, accept=True):
    """Manipular alerta"""
    alert = self.wait.until(EC.alert_is_present())
    if accept:
        alert.accept()

```

```

        else:
            alert.dismiss()

    def close(self):
        """Fechar driver"""
        self.driver.quit()

# USO
if __name__ == "__main__":
    bot = RPAAutomation()

    try:
        # Exemplo de fluxo RPA
        bot.navigate_to("https://exemplo.com/formulario")

        dados = {
            'nome': 'João Silva',
            'email': 'joao@email.com',
            'cargo': 'Analista',
            'aceito': True
        }

        bot.fill_form(dados)
        bot.take_screenshot("form_preenchido")

        print("✔ Processo RPA concluído!")

    finally:
        bot.close()

```

## RESUMO DOS 50 MAIS USADOS EM RPA

1. `driver.get(url)` - Acessar página
2. `find_element(By.ID, "id")` - Localizar elemento
3. `element.click()` - Clicar
4. `element.send_keys(texto)` - Digitar
5. `element.text` - Obter texto
6. `WebDriverWait + EC.element_to_be_clickable` - Espera inteligente

7. `driver.switch_to.window()` - Mudar entre janelas
8. `driver.switch_to.alert` - Manipular alertas
9. `Select(element)` - Trabalhar com dropdowns
10. `driver.execute_script()` - Executar JavaScript
11. `driver.save_screenshot()` - Capturar tela
12. `driver.refresh()` - Atualizar página
13. `element.clear()` - Limpar campo
14. `element.get_attribute()` - Obter atributo
15. `element.is_displayed()` - Verificar visibilidade
16. `ActionChains(driver)` - Ações complexas
17. `driver.back()` / `driver.forward()` - Navegação
18. `driver.current_url` - URL atual
19. `driver.title` - Título da página
20. `driver.find_elements()` - Múltiplos elementos
21. `element.submit()` - Submeter formulário
22. `driver.maximize_window()` - Maximizar
23. `driver.set_window_size()` - Definir tamanho
24. `driver.quit()` - Finalizar tudo
25. `element.is_enabled()` - Verificar habilitado
26. `element.is_selected()` - Verificar selecionado
27. `driver.implicitly_wait()` - Espera global
28. `driver.switch_to.frame()` - Trabalhar com iframes
29. `driver.switch_to.default_content()` - Sair do iframe
30. `driver.get_cookies()` - Manipular cookies
31. `driver.add_cookie()` - Adicionar cookie
32. `driver.delete_all_cookies()` - Limpar cookies
33. `driver.get_screenshot_as_base64()` - Screenshot base64
34. `driver.page_source` - Código fonte
35. `driver.set_page_load_timeout()` - Timeout de página
36. `element.screenshot()` - Screenshot de elemento
37. `NoSuchElementException` - Tratamento de erro
38. `EC.presence_of_element_located` - Espera presença
39. `EC.visibility_of_element_located` - Espera visibilidade
40. `EC.text_to_be_present_in_element` - Espera texto
41. `EC.title_contains` - Espera título
42. `Keys.ENTER` / `TAB` / `etc` - Teclas especiais



- 43. `chrome_options.add_argument()` - Configurações Chrome
- 44. `By.XPATH` - Localizador poderoso
- 45. `By.CSS_SELECTOR` - Localizador rápido
- 46. `actions.drag_and_drop()` - Arrastar e soltar
- 47. `actions.move_to_element()` - Mover para elemento
- 48. `select.select_by_visible_text()` - Selecionar dropdown
- 49. `driver.window_handles` - Gerenciar abas
- 50. `time.sleep()` - Espera fixa (usar com cautela)

## □ DICAS FINAIS PARA RPA COM SELENIUM:

1. **Sempre use waits explícitos** em vez de `time.sleep()`
2. **Prefira CSS Selectors** para performance (mais rápido que XPath)
3. **Use Page Object Pattern** para projetos grandes
4. **Faça screenshots** em pontos críticos para debugging
5. **Implemente retry logic** para elementos instáveis
6. **Mantenha os drivers atualizados** com webdriver-manager
7. **Use headless para produção**, mas mantenha visível para desenvolvimento
8. **Log tudo** - cada ação deve ser registrada
9. **Trate todos os exceptions** - RPA deve ser robusto
10. **Teste em diferentes resoluções** - responsividade importa